

Lab 02 - Runtimes (Part 2)

Instruction:

- This lab is divided into two parts. The objective of this part (Part 2) is to:
 - Relocate the `SelectionSort()` function (originally defined in Part 1) into a header file.
 - Define a function that determines whether an array is sorted.
 - Define a function that generates an array representing the worst-case input scenario for an ascending-order sorting algorithm.
 - Verify the accuracy of a simplified version of the runtime function derived in Part 1 of the `SelectionSort()`.
- Submit a `.h` source file and a `.cpp` source file.
- The file may include only these libraries: `iostream`, `string`, `vector`, `ctime`, `cstdlib` — no others.
- Submit the file to your GitHub repository, inside a directory named `Lab02`.
- Cheating of any kind is prohibited and will not be tolerated.
- **Violating and/or failing to follow any of the rules will result in an automatic zero (0) for the lab.**

At the very beginning of your submission, add a comment containing your name and the date, to confirm you have read and understood the instructions above.

Grading

Task	Maximum Points	Points Earned
1	1.0	
2	1.0	
3	0.5	
Total	2.5	

Task 1

Copy the `SelectionSort()` function and its runtime function (as a comment) from the `.cpp` source file of Part 1 into the header file. Next, define the function:

```
bool IsSorted(const vector<int>& v, bool d)
```

that returns `true` only if `v` is sorted in ascending order and `d` is `true`, or `v` is sorted in descending order and `d` is `false`; otherwise, it returns `false`. A vector with at most 1 element is considered sorted regardless of the value of `d`.

Task 2

Within the header file, define the function:

```
vector<int> WorstCase(size_t s)
```

that returns a vector of `s` randomly generated integers, arranged in descending order — this represents the worst-case input for an ascending-order sorting algorithm. If `s` is zero, an empty vector is returned.

Task 3

In the `.cpp` file, verify the proportionality of your `SelectionSort()` runtime function:

1. Declare two `time_t` objects, an integer vector object, and a `double` array of size 3.
2. Determine and store the time it takes `SelectionSort()` to terminate for array sizes 10000, 20000, and 40000, respectively. For each size:
 - a) Assign the vector object a newly generated array of the given size by invoking `WorstCase()`.
 - b) If the resulting vector is not sorted in descending order, display an error message and terminate the program.
 - c) Measure the execution time of `SelectionSort()` on the vector: record the time immediately before and immediately after invoking the function using the two `time_t` objects, and store the difference between them in the corresponding element of the `double` array (i.e., index 0 for size 10000, index 1 for size 20000, index 2 for size 40000).
3. Using the parent function of your runtime function from Part 1, calculate the theoretically expected time for the array sizes 20000 and 40000 relative to the measured time for size 10000, and compare each against its corresponding measured time (stored in the `double` array) to determine the percent error.

$$\text{Percent Error} = \left| \frac{\text{Experimental Time} - \text{Expected Time}}{\text{Expected Time}} \right| \times 100$$